
Alpha DQN: Adaptive Asset Allocation in Multi-Regime Financial Environments

Alberto Catellani

Department of Computer Science and Engineering (DISI)
University of Bologna
Bologna, Italy
alberto.catellani3@studio.unibo.it

Abstract

Sequential asset allocation under stochastic market conditions presents severe challenges due to high non stationarity, low signal to noise ratios, and delayed reward mechanisms. In this work, we propose *Alpha DQN*, an end to end reinforcement learning framework applied to a custom environment that models distinct corporate lifecycle behaviors and periodic dividend distributions. Crucially, to mitigate the widespread issue of overfitting to stochastic paths, we introduce an intentional low capacity neural network architecture designed to structurally regularize the agent. Our results demonstrate that enforcing strict capacity limits prevents the model from memorizing high frequency noise, successfully compelling it to discover robust macroeconomic heuristics such as asset regime rotation and dividend harvesting while consistently outperforming standard baselines.

1 Introduction

Algorithmic portfolio management remains a fundamental problem in quantitative finance, requiring continuous rebalancing of capital across diverse assets to maximize total returns while mitigating drawdown risk. Traditional frameworks, such as the seminal Markowitz mean-variance optimization, rely heavily on a decoupled two-step procedure: first estimating expected asset returns and covariances over historical windows, and subsequently solving a static convex optimization problem. As explained in recent comprehensive reviews of financial machine learning by Pippas et al. (2025) [1] and Bai et al. (2024) [2], this classical paradigm suffers from severe limitations. Most notably, traditional frameworks exhibit slow adaptability to structural regime shifts, utilize statistical optimization metrics that are heavily misaligned with downstream risk performance metrics, and incorporate execution constraints or transaction costs in an overly rigid, static manner [1, 2].

To address these shortcomings, Deep Reinforcement Learning (DRL) has emerged as an alternative, offering an agile, single step framework rooted in the foundational principles of Markov Decision Processes [3], where an autonomous agent maps environmental observations directly to portfolio actions by maximizing long term cumulative risk adjusted rewards [1, 2]. Value based methods, particularly variants of the Deep Q-Network (DQN) originally introduced by Mnih et al. [4], have achieved substantial prominence within the portfolio management literature due to their sample efficiency and capability to approximate complex, non-linear action-value structures in discrete action spaces [1, 2].

However, applying DRL to empirical financial data introduces profound algorithmic vulnerabilities. Financial time series are inherently noisy, non stationary, and prone to heavy tailed distributions [1, 2]. When standard high capacity deep networks are exposed to historical market data, they frequently succumb to severe overfitting, memorizing localized stochastic patterns inside the experience replay buffer rather than isolating persistent economic signals [1]. Furthermore, validating the true strategic

rationale of an agent’s policy is notoriously difficult in empirical environments due to unobservable latent factors and variable data frequencies [2].

2 Custom Market Environment Architecture

To rigorously test the policy synthesis capabilities of value based reinforcement learning, we designed an analytically transparent, discrete time financial simulator named *QuantamentalEnv*. The environment models a finite horizon sequence of trading intervals where an agent must allocate capital among distinct corporate archetypes and a risk free cash account, subject to capital distributions and stochastic price drift.

2.1 Asset Price and Dividend Dynamics

The investment universe contains $N = 3$ distinct risky equities, each corresponding to a specific stylized phase of a corporate lifecycle: High Growth (GROW), Stable Value (VALU), and Corporate Decline (DECL). The mid-price path $P_i(t)$ for each asset $i \in \{1, \dots, N\}$ is modeled as a discrete time Geometric Brownian Motion (GBM):

$$P_i(t+1) = P_i(t) \exp \left(\left(\mu_i - \frac{\sigma_i^2}{2} \right) \Delta t + \sigma_i \sqrt{\Delta t} Z_{i,t} \right) \quad (1)$$

where μ_i represents the structural expected asset drift, σ_i dictates the asset volatility, $\Delta t = 1$ is the unit time step (one simulated trading day), and $Z_{i,t} \sim \mathcal{N}(0, 1)$ are independent and identically distributed standard normal random variables.

In addition to price fluctuations, assets periodically distribute capital via dividends. Each asset is initialized with a structural baseline payout ratio and dividend yield. To simulate real world financial noise and execution uncertainty, the distributed dividend at an interval t is perturbed by additive Gaussian noise governed by a standard deviation parameter $\sigma_d = 0.10$.

Table 1 summarizes the empirical calibrations for each corporate lifecycle archetype.

Table 1: Asset lifecycle calibration parameters.

Asset	μ	σ	Payout	Yield
GROW (High Growth)	+0.0018	0.005	10%	0.2%
VALU (Stable Value)	+0.00005	0.002	60%	2.5%
DECL (Corporate Decline)	−0.0015	0.004	80%	1.0%

2.2 State Space $\mathcal{S}$

To ensure a fully Markovian representation where the current observation contains all necessary information for optimal decision-making, the state space is designed to encapsulate both localized asset dynamics and private portfolio metrics within a single continuous vector of dimension $D_{\text{obs}} = 16$. The state vector $\mathbf{s}_t \in \mathcal{S}$ at any given interval is partitioned as follows:

$$\mathbf{s}_t = \left[\mathbf{f}_1(t), \mathbf{f}_2(t), \mathbf{f}_3(t), c_t, w_1(t), w_2(t), w_3(t) \right]^T \quad (2)$$

- Public Asset Features** ($\mathbf{f}_i(t) \in \mathbb{R}^4$): For each company $i \in \{1, \dots, N\}$, the environment exposes four dynamic market indicators: the immediate localized log-return, a rolling historical volatility estimator, the current trailing dividend yield, and a deterministic temporal countdown tracking the remaining daily intervals until the next scheduled dividend distribution window.
- Private Portfolio States** ($\mathbb{R}^4$): Comprises the scalar unallocated cash balance c_t (stochastically initialized across episodes between \$50,000 and \$150,000 to mitigate initial capital dependency) alongside the N dimensional vector of active allocation weights $\mathbf{w}(t) = [w_1(t), w_2(t), w_3(t)]$, representing the current fraction of total wealth deployed in each equity.

2.3 Action Space $\mathcal{A}$

The action space is formulated as a discrete choice framework consisting of $M = 5$ structural asset allocation heuristics:

$$\mathcal{A} = \{a_1, a_2, a_3, a_{\text{cash}}, a_{\text{equal}}\} \quad (3)$$

When the agent selects an action $a \in \mathcal{A}$, the simulator executes a programmatic portfolio rebalancing script at the start of interval $t + 1$:

- Actions a_i for $i \in \{1, 2, 3\}$ correspond to a complete allocation (100% of capital) directly into asset i , liquidating all other structural dependencies.
- Action a_{cash} liquidates all equity positions to hold 100% of total wealth in the risk free cash account.
- Action a_{equal} enforces a naive diversification across all three active risky assets.

2.4 Portfolio Accounting and Dividend Mechanics

Let c_t and $n_{i,t}$ denote the cash balance and share quantity in asset i at step t . The nominal Net Asset Value is:

$$NAV_t = c_t + \sum_{i=1}^N n_{i,t} P_i(t) \quad (4)$$

Rather than tracking smooth continuous cash distributions, our environment simulates structural corporate payouts by executing discrete dividend events at the conclusion of every 90 step interval ($K = 90$), replicating a standard quarterly earnings cycle. When the deterministic temporal countdown feature $\tau_{i,t}$ reaches zero, a corporate distribution is triggered. The distributed dividend per share $D_{i,t}$ for asset i is stochastically computed based on its current market price, its historical baseline dividend yield Y_i , and an additive normal noise factor truncated at zero to guarantee non negative cash flows:

$$D_{i,t} = P_i(t) \cdot Y_i \cdot \max(1.0 + \epsilon_{i,t}, 0.0), \quad \epsilon_{i,t} \sim \mathcal{N}(0, \sigma_d^2) \quad (5)$$

where $\sigma_d = 0.10$ represents the volatility of the dividend noise.

If the agent holds a positive equity position ($n_{i,t} > 0$) at the moment of distribution, the aggregate dividend payment is instantaneously injected directly into the agent's liquid cash account:

$$c_t \leftarrow c_{t-1} + \sum_{i=1}^N n_{i,t} D_{i,t} \quad (6)$$

Following the distribution event, the countdown timer for the asset is reset back to its baseline value ($\tau_{i,t+1} = K$).

2.5 Inflation Adjustment and Reward Formulation

To prevent the agent from optimizing nominal growth at the expense of purchasing power, we discount the portfolio by a daily inflation rate π , defining the real NAV as:

$$NAV_t^{\text{real}} = \frac{NAV_t}{(1 + \pi)^t} \quad (7)$$

To maintain a completely scale-invariant feedback loop that remains robust against randomized initial capital structures (which vary stochastically between \$50,000 and \$150,000), the immediate feedback reward R_t delivered to the DQN agent at step t is formulated as the fractional rate of return of this inflation-adjusted real asset value:

$$R_t = \frac{NAV_t^{\text{real}} - NAV_{t-1}^{\text{real}}}{NAV_{t-1}^{\text{real}}} \quad (8)$$

By expanding this fraction using our real valuation definition, the immediate reward signal can be cleanly expressed as a function of nominal changes penalized by the single period inflation rate:

$$R_t = \frac{NAV_t}{NAV_{t-1}(1 + \pi)} - 1 \quad (9)$$

3 Deep Q-Network Agent Design

We implement a value based DQN agent whose primary architectural objective is structural regularization via capacity constraints.

3.1 Network Architecture and Parameter Bounds

The state-action value function $Q(s, a; \theta)$ is approximated using a feedforward Multi Layer Perceptron (MLP). The network maps the continuous 16 dimensional observation vector $\mathbf{s}_t \in \mathbb{R}^{16}$ to the 5 dimensional discrete action space $\mathcal{A} \in \mathbb{R}^5$ through two fully connected hidden layers containing exactly 64 hidden units each, utilizing Rectified Linear Unit (ReLU) activation functions

Crucially, rather than deploying deep, high capacity representations that are standard in vision or control tasks, we restrict the network’s capacity. The layer configurations yield a total learnable parameter allocation calculated as:

$$\mathcal{P} = \underbrace{(16 \times 64 + 64)}_{\text{Layer 1: 1,088}} + \underbrace{(64 \times 64 + 64)}_{\text{Layer 2: 4,160}} + \underbrace{(64 \times 5 + 5)}_{\text{Layer 3: 325}} = 5,573 \text{ parameters} \quad (10)$$

Limiting the architecture to exactly 5,573 parameters serves as a robust bottleneck constraint. Because the network lacks the capacity to memorize individual high frequency price paths inside the experience replay buffer, it is mathematically compelled to isolate structural, macro economic invariants that persist across independent simulation paths, such as co-movements, drift signs, and explicit temporal countdown triggers.

To ensure stable gradient propagation at the inception of training, weights are initialized using Kaiming Uniform initialization. This guarantees variance stability across layers before any optimizer updates are applied.

3.2 Value Function Optimization

The agent learns by minimizing the mean squared Bellman error over mini batches of transitions extracted from an Experience Replay Buffer $\mathcal{D}$ with a capacity of $|\mathcal{D}| = 100,000$ steps. To stabilize value function updates against the "moving target" problem inherent to standard Q-learning, we leverage the stabilizing mechanisms established by Mnih et al. [4]: a uniform experience replay protocol and a decoupled target network parameterized by θ^- , which is updated via hard synchronization updates every $C = 1,000$ global training steps.

For a mini batch of transitions $B = \{(s_j, a_j, R_j, s_{j+1}, d_j)\}_{j=1}^{|B|}$ sampled uniformly from $\mathcal{D}$, the temporal difference (TD) target Y_j is defined as:

$$Y_j = R_j + \gamma(1 - d_j) \max_{a' \in \mathcal{A}} Q(s_{j+1}, a'; \theta^-) \quad (11)$$

where $\gamma = 0.99$ represents the temporal discount factor and $d_j \in \{0, 1\}$ is the termination flag indicating if s_{j+1} is a terminal step.

3.3 Optimization Strategy and Loss Regularization

The online network θ is updated via the Adam optimizer with learning rate $\alpha = 0.00025$. To prevent erratic gradient updates from sudden dividend-induced NAV spikes or multi sigma price shocks, we adopt the Huber (Smooth L1) loss:

$$L(\theta) = \frac{1}{|B|} \sum_{j \in B} \mathcal{L}_\delta(Y_j - Q(s_j, a_j; \theta)), \quad \mathcal{L}_\delta(x) = \begin{cases} 0.5 x^2 & |x| \leq 1 \\ |x| - 0.5 & \text{otherwise} \end{cases} \quad (12)$$

The quadratic regime ensures smooth convergence on small TD errors, while the linear regime caps gradient magnitudes for large outliers, preventing value-function corruption during extreme market events.

4 Empirical Evaluation and Experimental Results

To validate the core thesis that structural capacity regularizes a value based reinforcement learning agent against stochastic noise, we conduct a series of empirical evaluation experiments. The agent is trained over 2,000 episodes, and the synthesized policy is subsequently evaluated across 6,000 completely independent market trajectories to guarantee statistical significance and ensure zero data leakage across random walks.

4.1 Experimental Setup and Baselines

The performance of the trained *Alpha DQN* agent is benchmarked against an unguided **Random Allocation Baseline** agent across a fixed evaluation horizon of $T = 252$ trading steps (representing a standard corporate trading year). At each interval t , the random baseline samples an action uniformly at random from $\mathcal{A}$. This baseline measures the expected performance of an unguided, high turnover trading policy within our custom geometric Brownian motion constraints, serving as a rigorous null hypothesis to test if the agent has decoded the underlying macroeconomic dynamics.

4.2 Single-Episode Trajectory Analysis

To investigate the strategic rationale of the learned policy, Figure 1 details the deterministic step by step telemetry captured during a representative evaluation episode. The trajectory demonstrates that the capacity constrained agent successfully filters out high frequency noise to exploit underlying macroeconomic mechanics.



Figure 1: Comparison of DQN and random agent evaluation at episode 100.

By analyzing the alignment between price shifts and allocation weight mutations over time, several advanced financial heuristics emerge autonomously:

1. **Structural Decline Avoidance:** The agent maintains near zero allocation to DECL throughout the episode, even during transient stochastic upswings, confirming that the 5,573 parameter bottleneck isolates the negative structural drift ($\mu_{\text{DECL}} = -0.0015$) rather than reacting to localized noise.
2. **Opportunistic Dividend Harvesting:** As the countdown feature $\tau_{\text{VALU},t}$ approaches zero near quarterly boundaries ($t \in \{90, 180, 270\}$), the agent rotates from GROW into VALU to capture the high yield distribution ($Y_{\text{VALU}} = 2.5\%$), then reverts to growth exposure immediately after the payout.
3. **Risk-Mitigating Diversification:** During high uncertainty windows or multi sigma draw-downs, the agent shifts to a_{equal} as a transient hedge until directional trends stabilize.

4.3 Comparative Performance Summary

To provide a rigorous quantitative validation of the architecture’s generalization capability, Table 2 presents the empirical results aggregated over 6,000 independent evaluation trajectories.

Table 2: Statistical Performance Metrics Across 6,000 Independent Test Trajectories

Strategy	Test Episodes	Mean Return	Std. Dev. Return	Sharpe Ratio	Max Drawdown
Random Baseline	6,000	4.446%	5.718%	0.4648	−4.627%
Alpha DQN	6,000	24.706%	10.050%	3.1459	−2.402%

Alpha DQN achieves a mean cumulative return of 24.706%, outperforming the Random Baseline (4.446%) by an absolute margin of 20.26%, with a Sharpe Ratio of 3.1459 versus 0.4648 and a mean maximum drawdown of −2.402% versus −4.627%. These results confirm that a low capacity architectural bottleneck paired with an informative Markov state forces the agent to filter stochastic path noise and synthesize generalizable market Alpha.

5 Conclusion and Future Work

We introduced *Alpha DQN*, a value based RL framework for multi regime asset allocation. By restricting the MLP to 5,573 parameters, we demonstrated that architectural capacity limits serve as an effective inductive bias: the agent cannot memorize stochastic price paths and is instead compelled to isolate persistent macroeconomic invariants negative drift avoidance and dividend harvesting that generalize across independent trajectories.

While these results provide a strong validation of low capacity structural regularization, several critical dimensions remain open for future investigation to bridge the gap between stylized simulations and live-market execution:

- **Continuous Allocation Spaces:** The current discrete action set collapses fine grained portfolio weights into five macro-actions. Future work will explore actor-critic architectures to output continuous weight vectors over the allocation simplex.
- **Transaction Costs and Liquidity Frictions:** The environment currently assumes zero execution slippage. Integrating realistic linear or quadratic trading cost functions will force the agent to weigh expected alpha against liquidation drag, yielding more practically deployable policies.
- **Latent Regime Modeling:** GBM drift parameters μ_i remain stationary within an episode. Introducing hidden Markov macro regime switches stochastically alternating between expansionary and contractionary drift regimes would provide a rigorous stress test for out of distribution generalization.

References

- [1] N. Pippas, E. A. Ludvig, and C. Turkay. The Evolution of Reinforcement Learning in Quantitative Finance: A Survey. *ACM Computing Surveys*, 57(11), Article 295, June 2025.
- [2] Y. Bai, Y. Gao, R. Wan, S. Zhang, and R. Song. A Review of Reinforcement Learning in Financial Applications. *arXiv preprint arXiv:2411.12746v1*, November 2024.
- [3] R. S. Sutton and A. G. Barto. *Reinforcement learning: An introduction*. MIT press, second edition, 2018.
- [4] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, February 2015.